University of Sialkot

Semester Project – (Client-Side Architecture)

Submitted by: Sabeeha Naveed (23010101-131)
Muneeba (23010101-052)

Department: Software engineering

Section: Blue

Subject: Web Engineering

Project Title: Medicine Reminder App

Submitted to: Sir Museb

Assignment no. 03

Introduction

The **Medicine Reminder App** is a client-side web application developed using **Vue 3** and **Vite**. The main objective of the application is to help users manage their daily medication schedule by allowing them to add medicines, set reminder times, and view their medicine schedule in an organized interface.

2. Technologies Used

- Vue.js 3
- Vite
- Vue Router
- components
- Props
- Emitters
- CSS

Project Structure

The project is divided into two main folders:

Views

The **Views** folder contains all the main pages of the application.

- Splash.vue
- Login.vue
- Signup.vue
- Dashboard.vue
- AddMedicine.vue
- Schedule.vue
- History.vue
- Profile.vue
- EditProfile.vue
- Settings.vue

Components

The **Components** folder contains reusable UI components.

- Sidebar.vue
- MedicineCard.vue
- ProfileCard.vue
- SettingItem.vue

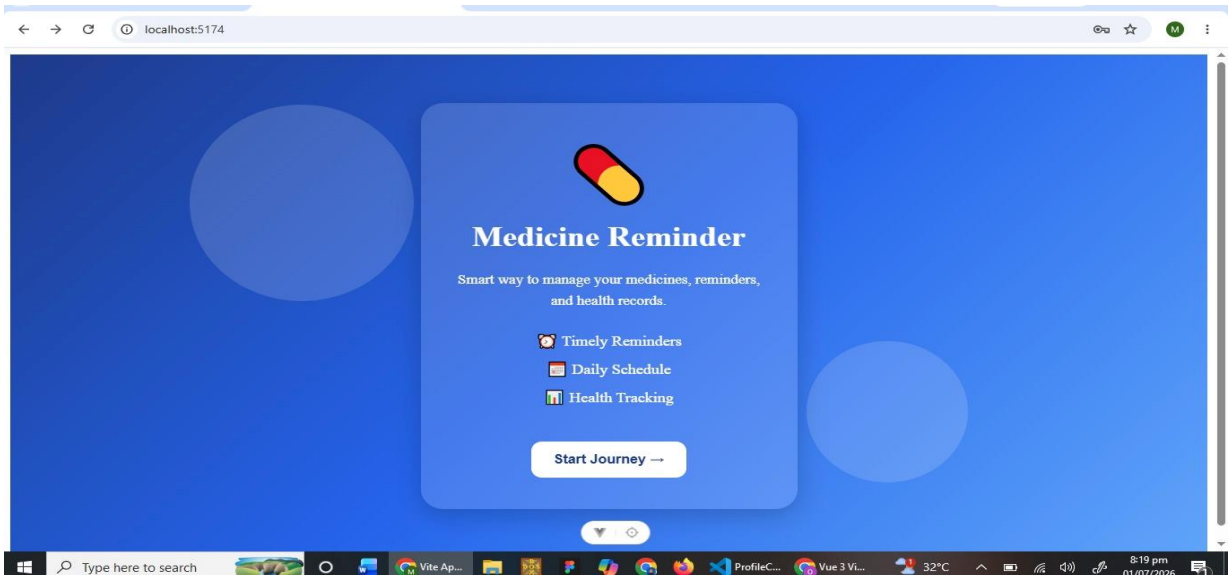
Pages Explanation

1. Splash Screen

The Splash Screen is the first page displayed when the application starts.

Purpose:

- Displays the app logo and name.
- Redirects the user to the Login or Signup page.



2. Login Page

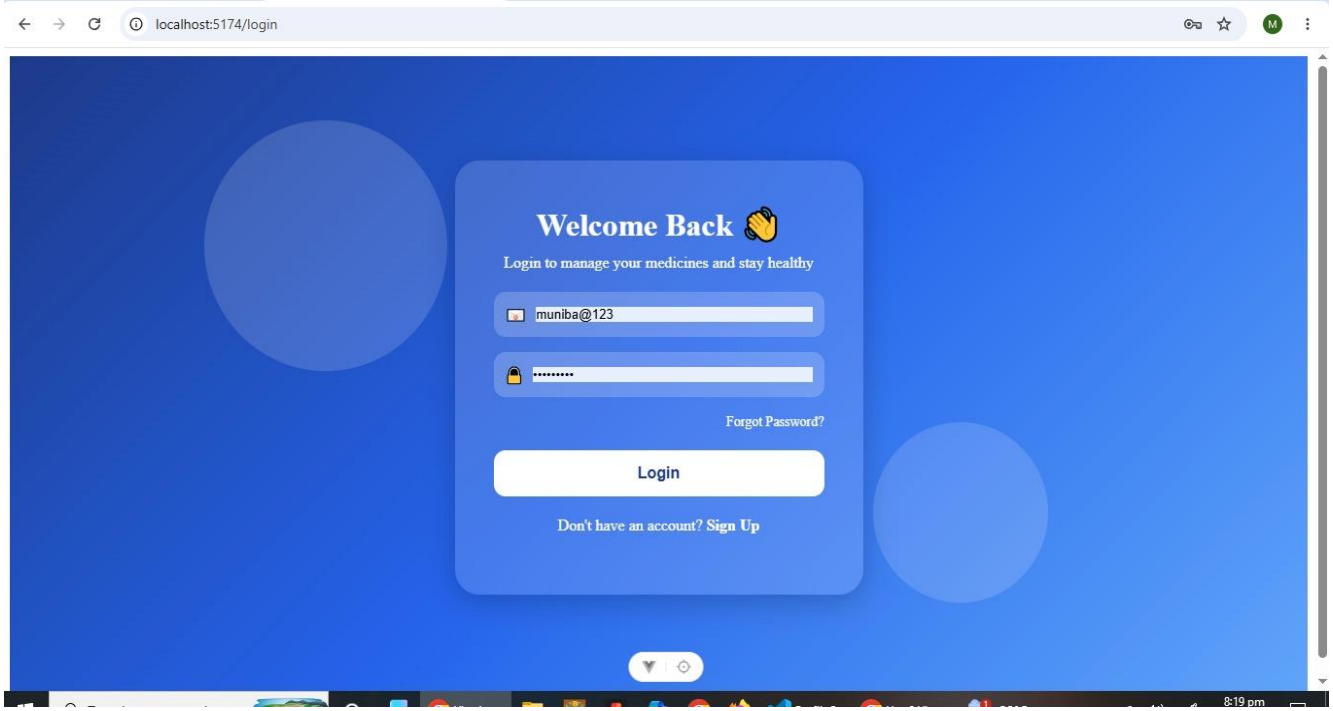
The Login page allows existing users to access the application.

Fields:

- Email
- Password

Button:

- Login



3. Signup Page

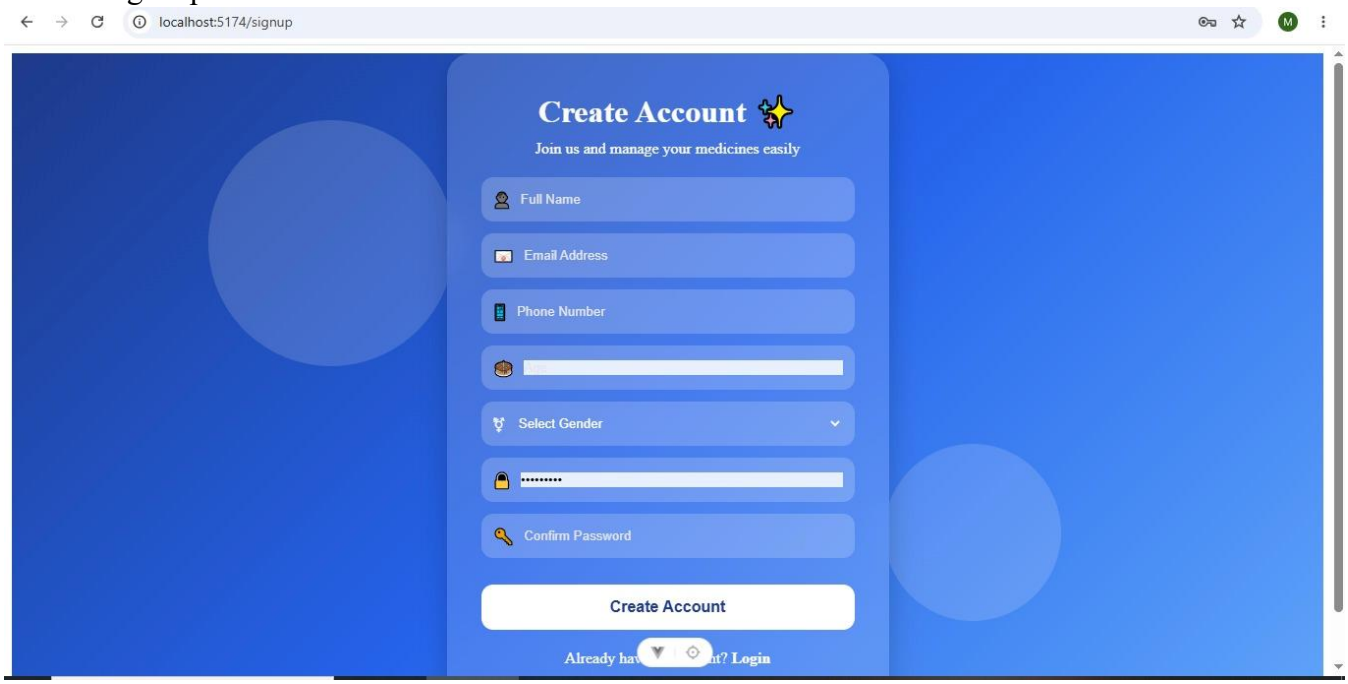
The Signup page allows new users to create an account.

Fields:

- Full Name
- Email
- Password
- gender

Button:

- Sign Up



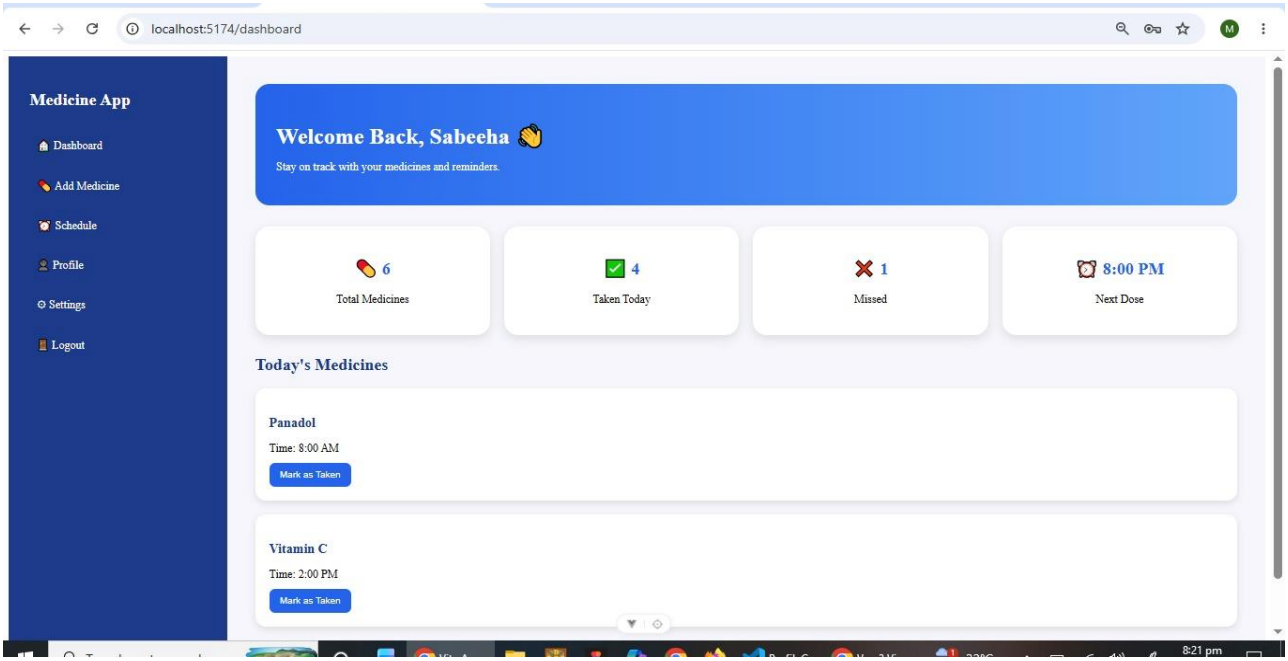
4. Dashboard

The Dashboard is the main home page after login.

It displays:

- Welcome message
- Total medicines
- Taken medicines
- Missed medicines
- Next medicine reminder

The **MedicineCard** component is used to display medicine information.



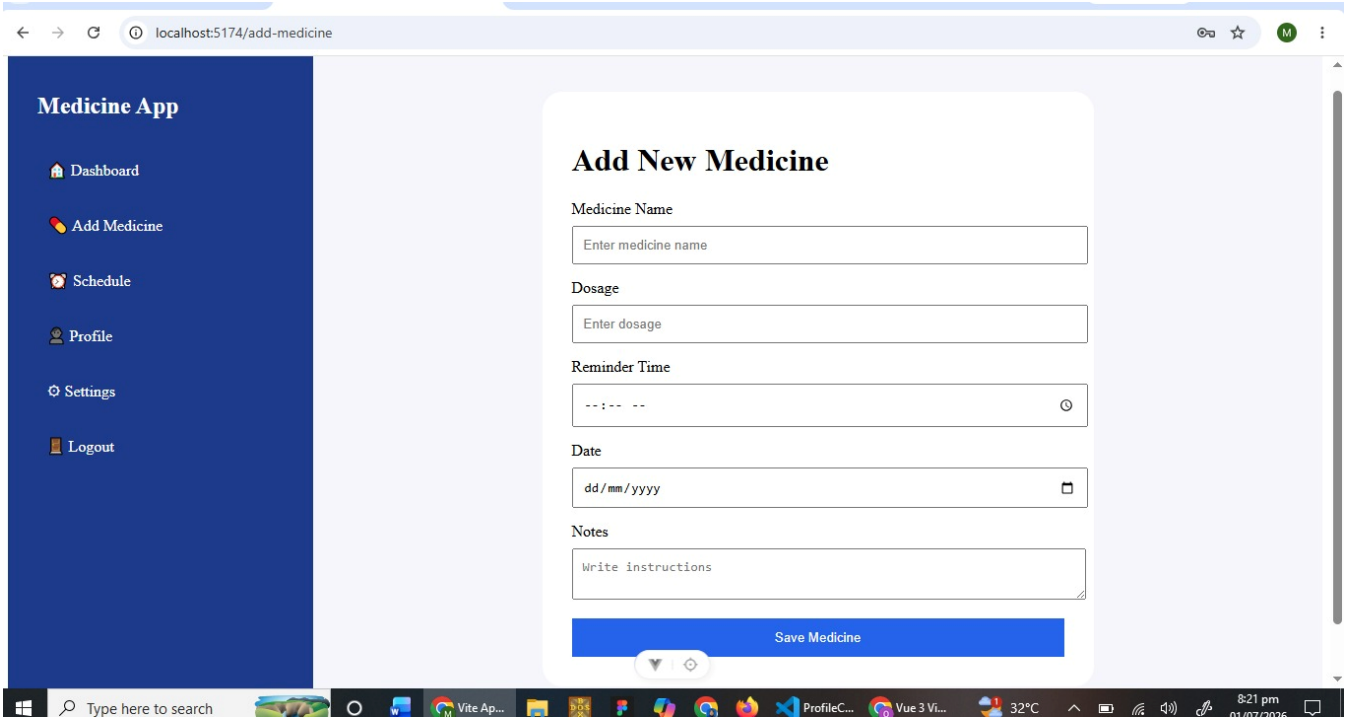
5. Add Medicine

This page allows users to add a new medicine.

Fields:

- Medicine Name
- Dose
- Reminder Time
- Date

Users can save the medicine for future reminders.



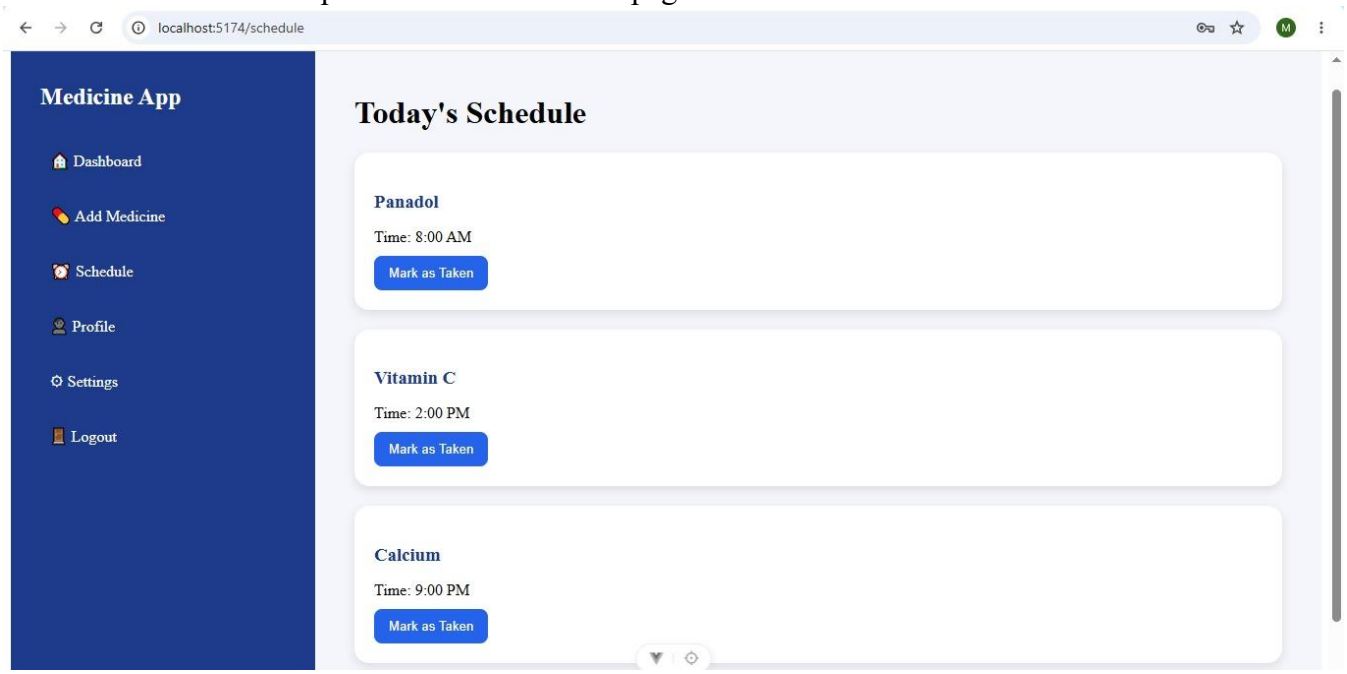
6. Schedule Page

The Schedule page displays all daily medicine reminders.

Users can:

- View medicine details
- Mark a medicine as taken

The **MedicineCard** component is reused on this page.



7. History Page

The History page keeps a record of previous medicines.

It shows:

- Taken medicines
- Missed medicines
- Date and time history

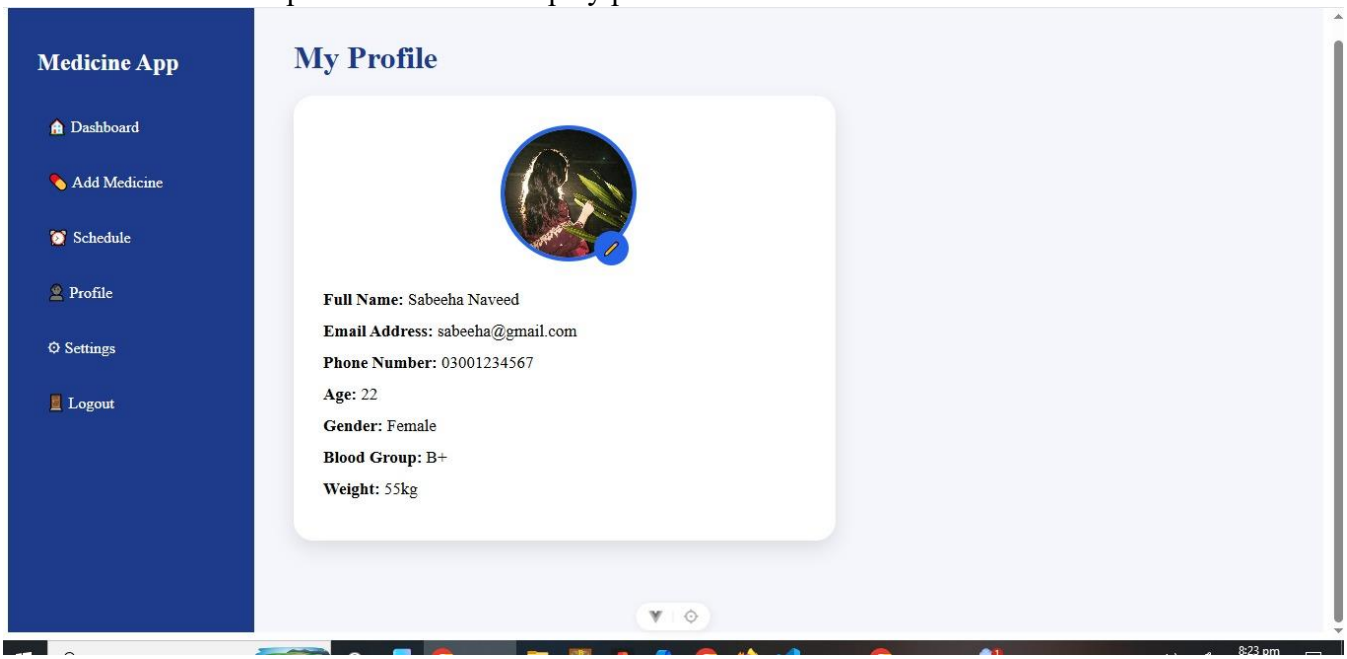
8. Profile Page

The Profile page displays user information.

Information includes:

- Name
- Email
- Phone Number
- Age
- Gender
- Blood Group
- Weight

The **ProfileCard** component is used to display profile information.



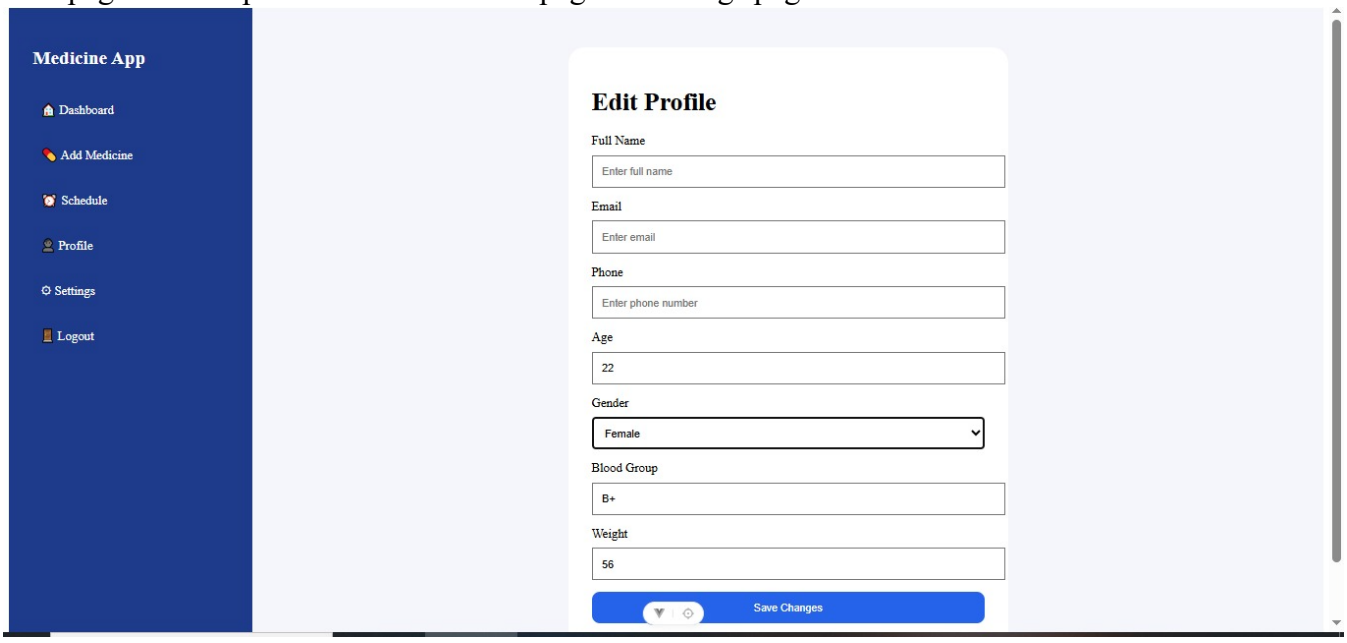
9. Edit Profile Page

This page allows users to update their profile information.

Users can edit:

- Name
- Phone Number
- Age
- Gender
- Blood Group
- Weight

This page can be opened from the Profile page or Settings page.



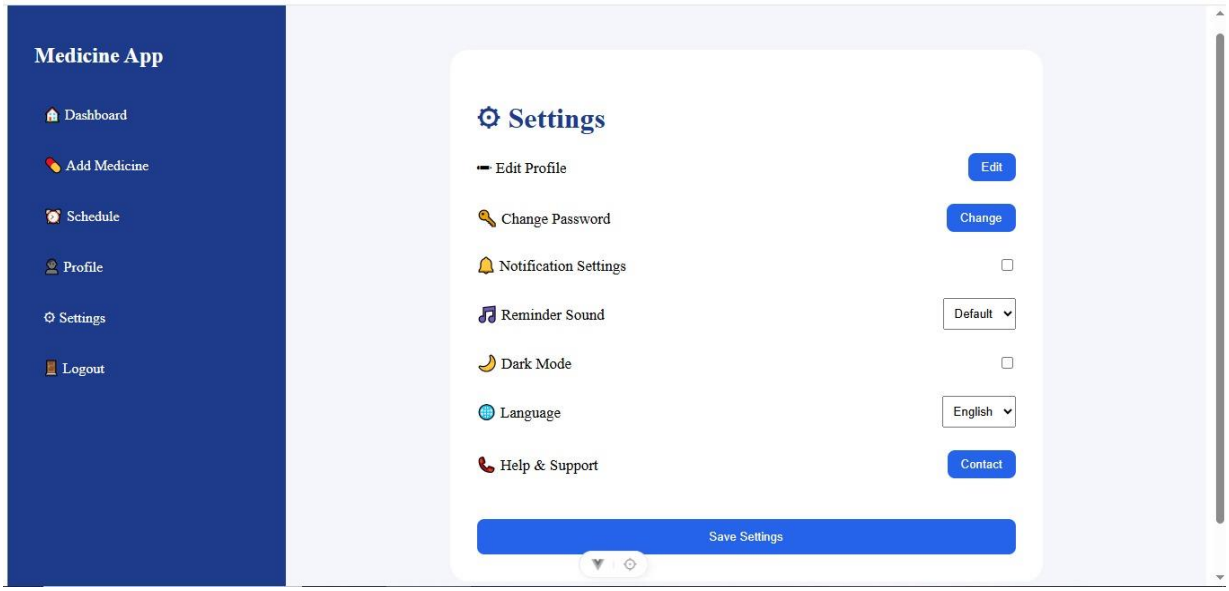
10. Settings Page

The Settings page allows users to customize the application.

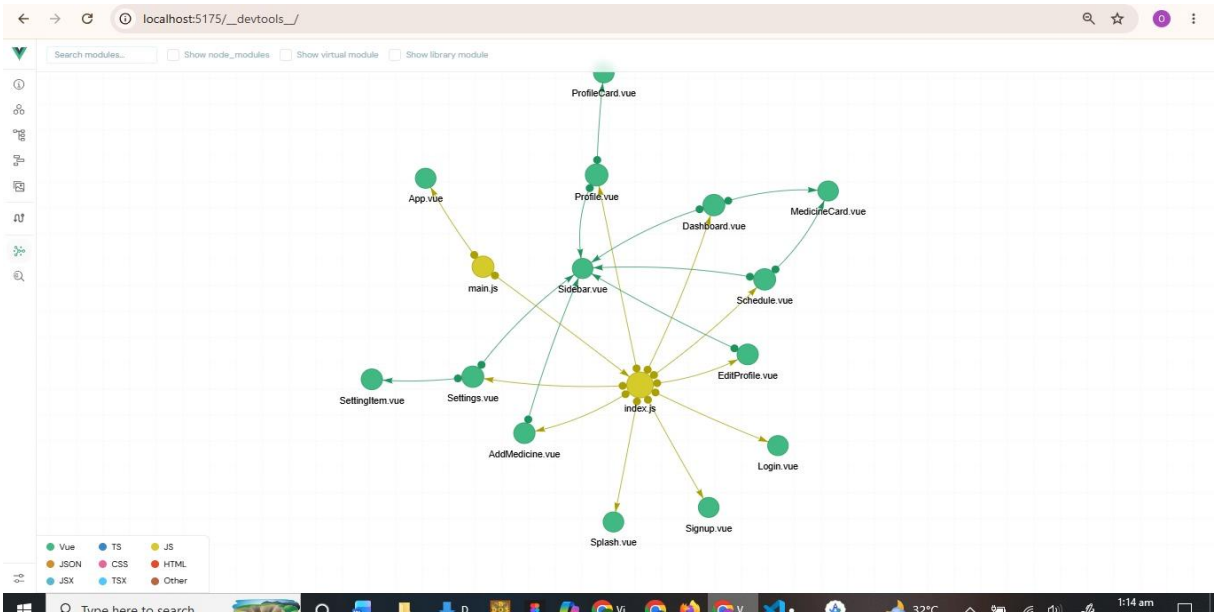
Features include:

- Edit Profile
- Change Password
- Notification Settings
- Reminder Sound
- Dark Mode
- Language
- Help & Support

The **SettingItem** component is used for each setting option.



Architecture Tree



Code Snippets

Props

Props are used to pass data from a parent component to a child component.

Flow:

Parent → Child

MedicineCard props:

```
const props = defineProps({
  medicineName: String,
  medicineTime: String
})
```

ProfileCard props:

```
defineProps({
  name: String,
  email: String,
  phone: String,
  age: Number,
  gender: String,
  bloodGroup: String,
  weight: String
})
```

Setting props:

```
const props = defineProps({
  title: String,
  type: String,
  buttonText: String,
  link: String,
  options: Array
})
```

Sidebar props:

```
defineProps({
  title: String
})
```

Emitters

Emitters are used to send events from a child component back to the parent component.

Flow:

Child → Parent

MedicineCard Emitters:

```
const emit = defineEmits(['medicineTaken'])

const markTaken = () => {
  emit('medicineTaken', props.medicineName)
}
```

ProfileCard Emitts:

```
const emit = defineEmits(['editClicked'])

const editProfile = () => {
  emit('editClicked')
}
```

Setting Emitter:

```
const router = useRouter()

const props = defineProps({
  title: String,
  type: String,
  buttonText: String,
  link: String,
  options: Array
})

const emit = defineEmits(['settingChanged'])

const handleClick = () => {
  emit('settingChanged')

  // popup force show
  alert("Setting updated successfully")

  // after popup close, navigate
  if (props.link) {
    router.push(props.link)
  }
}

const sendEvent = () => {
  emit('settingChanged')
  alert("Setting updated successfully")
}
```